EXPERIMENT NO: 07

ROLL No:

NAME:

TITLE: Porting of FreeRTOS to Arduino/STM32.

Apparatus: Arduino Board, Computer,FreeRTOS Source Code, Arduino IDE, LED, Resistor, Virtual Terminal, potentiometer and Proteus software.

Theory:
Real time operating system popularly known as RTOS provides controller with the ability to respond to input and complete tasks within a specific period of time based on priority. On the first look, an RTOS might sound like just any other embedded program or firmware, but it is built on the architecture of an Operating system. Hence, like any operating system RTOS can allow multiple programs to execute at the same time supporting **multiplexing**. As we know the core of a processor or controller can only execute a single instruction at a time, but the RTOS has something called the **scheduler** which decides which instruction to execute first and thus executes the instructions of multiple programs one after the other. Technically an RTOS only creates an illusion of multi-taking by executing paralleled instructions one at a time.

Code:
Arduino code of two task analogeread and blink task:

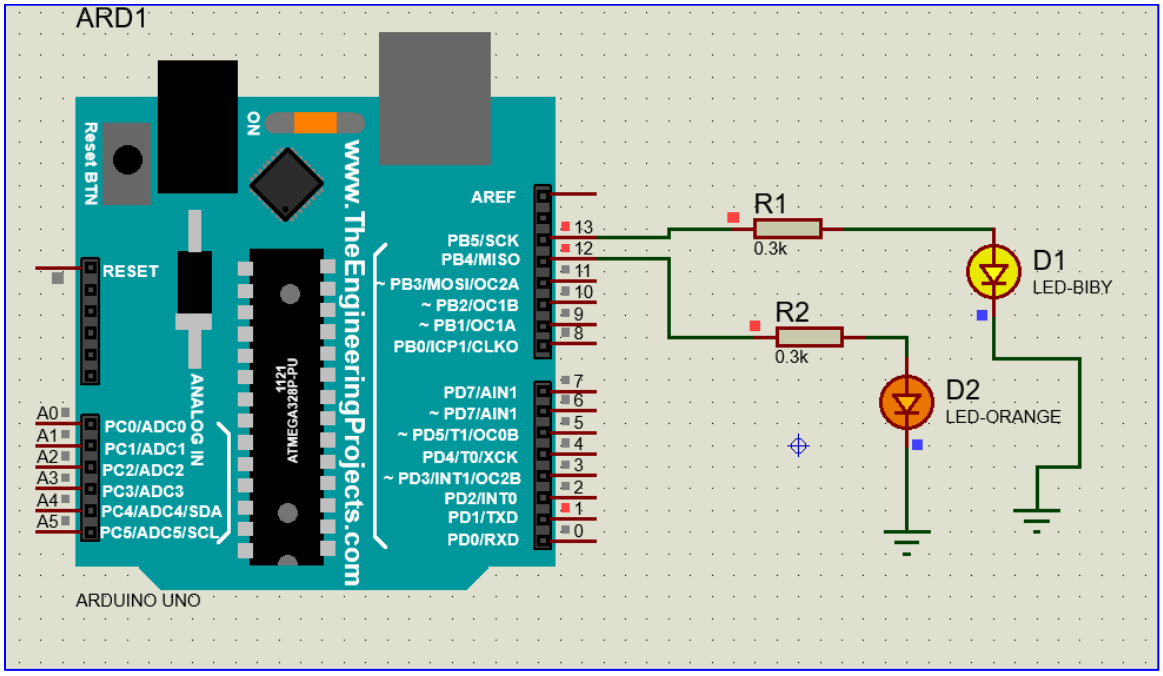
```
#include <Arduino_FreeRTOS.h>

// Define the pins for the LEDs
#define LED1 13
#define LED2 12

// Task handles (handler will do such as suspending, resuming, or deleting
                the task.)
//For example, if you wanted to suspend Task 1 at some point in the
program, we could use vTaskSuspend(Task1_Handle);

TaskHandle_t Task1_Handle;
TaskHandle_t Task2_Handle;
```

```
// Task functions
void Task1Blink(void *pvParameters);
void Task2Blink(void *pvParameters);

void setup() {

  // Set the LED pins as outputs
  pinMode(LED1, OUTPUT);
  pinMode(LED2, OUTPUT);

  // Create Task 1                                                //1=Priority
  xTaskCreate(Task1Blink, "Task1", 128, NULL, 1, &Task1_Handle);   //128 is
                                                                  stack
                                                                  size(memory
                                                                  )

  // Create Task 2
  xTaskCreate(Task2Blink, "Task2", 128, NULL, 1, &Task2_Handle);

  // Start the scheduler
  vTaskStartScheduler();
}

void loop() {
  // Empty loop since tasks are running in FreeRTOS scheduler
}

// Task 1: Blinks LED1 at a rate of 500ms
void Task1Blink(void *pvParameters) {
  (void) pvParameters;  //This prevents the compiler from generating
warnings about the unused parameter, helping keep the code clean and
warning-free.


  while (1) {
    digitalWrite(LED1, HIGH);
    vTaskDelay(500 / portTICK_PERIOD_MS);
    digitalWrite(LED1, LOW);
    vTaskDelay(500 / portTICK_PERIOD_MS);
  }
}

// Task 2: Blinks LED2 at a rate of 1000ms
void Task2Blink(void *pvParameters) {
```

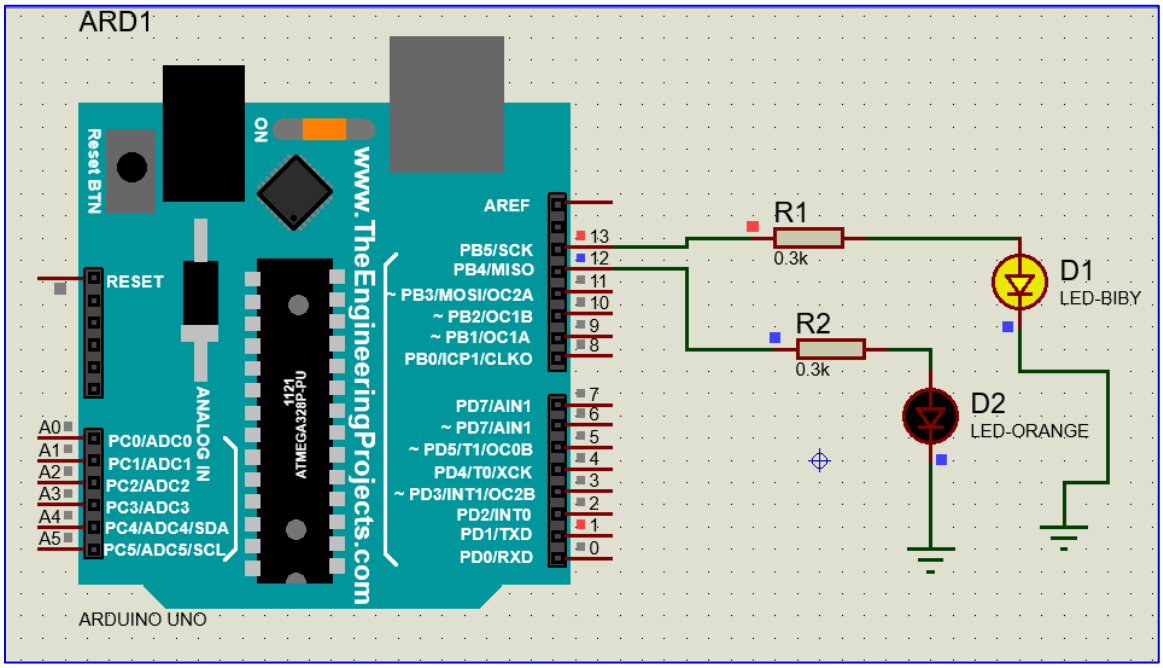
```
  (void) pvParameters;
  while (1) {
    digitalWrite(LED2, HIGH);
    vTaskDelay(1000 / portTICK_PERIOD_MS);
    digitalWrite(LED2, LOW);
    vTaskDelay(1000 / portTICK_PERIOD_MS);
  }
}
```

Result:

1)



2)



Conclusion: The successful porting of FreeRTOS to the Arduino platform expands the capabilities of Arduino by providing with a powerful real-time operating system for building more sophisticated embedded systems. With FreeRTOS integrated into the Arduino ecosystem, we can leverage multitasking and real-time scheduling to create responsive and efficient applications for a wide range of projects. This experimnets lays the foundation for further exploration and innovation in the field of embedded systems development with Arduino.